Lecture 1: Boosting

Alex Reinhart

Machine Learning II: 46-927

January 25–27, 2021

Announcements

- Homework 1 out soon, due next week (February 3).
- We will be setting up office hours shortly.
- Project teams and initial ideas (discussed below) are also due by next week (February 3).

Class content

- Finishing the classic Stat/ML topics: Boosting, Support Vector Machines.

- Special topics
  - Unsupervised methods: Clustering, dimension reduction, topic models, mixture models.
  - Survival analysis, point processes (time permitting)
  - Deep learning
  - Reinforcement learning introduction

Class structure

For the most part, everything is very similar to last semester:

- Homework style
- Regular quizzes on Gradescope
- No final exam

However, **this mini there is also a group project**

Homework

Homework will be due by the start of each Wednesday class.

- I may move/cancel a homework near the end of term to make time for project work.

Please be sure to:

- Actually submit your homework on time. I will continue discussing solutions right after the deadline.
- Check to be sure you actually submitted the right thing. Just open it after submission and check!

Final Project

The project is rather open-ended. You have essentially three possible directions:

- Reproduce/extend the results of a finance ML paper, apply to a new area. Validate carefully and see if it actually works...
- Demonstrate techniques we have used carefully on a financial application
- More deeply explore a topic in ML and teach us about it.

There is a writeup on Canvas providing more detail on the project.

Final Project

For the project, you will deliver

- A report (8 pages **at most**) describing your work
- A short presentation about your work, given to the class and intended to teach them about the ML method or the financial application

Homeworks will gradually get shorter so you have time to work on the project.

Final Project: By next week!

By next week (February 3):

- Choose your team of 3–4 people.
- Brainstorm topics.
- By February 3, submit a short description of what you'd like to do (1 paragraph) on Canvas. You can submit several ideas if you want feedback, in case some are infeasible.

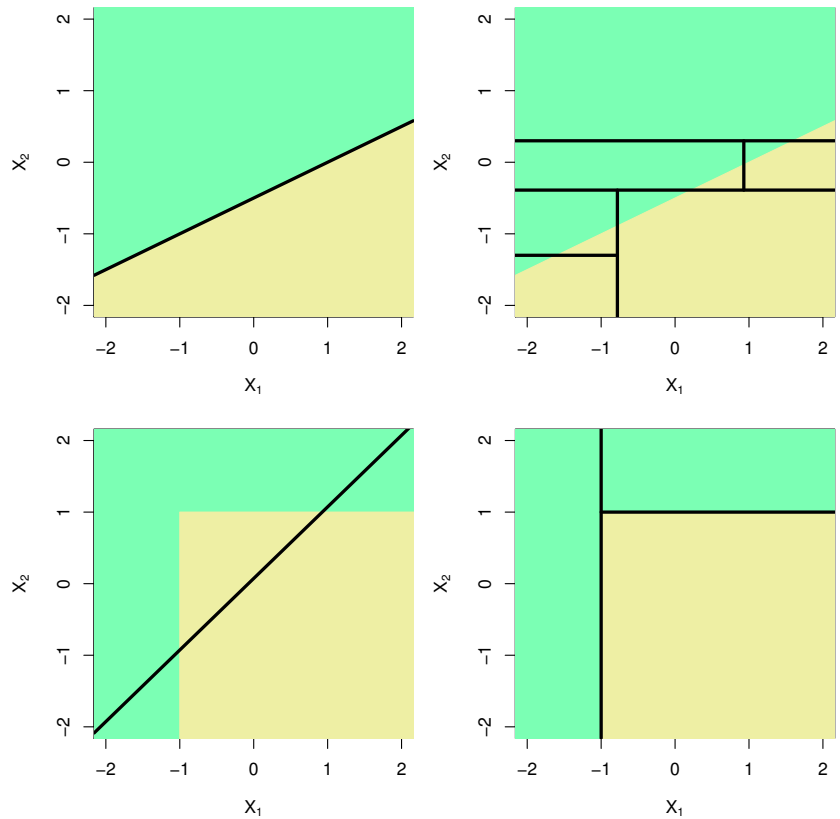
Today: Boosting

- Quick review of where we left off
- Boosting!
- Other ensemble methods?

Back to last mini: Trees

You learned about regression and decision trees in ML1.

These methods approximate $\mathbb{E}(Y|X)$ or $\mathbb{P}(Y = 1|X)$ by constant values on a rectangular partition of the X space.

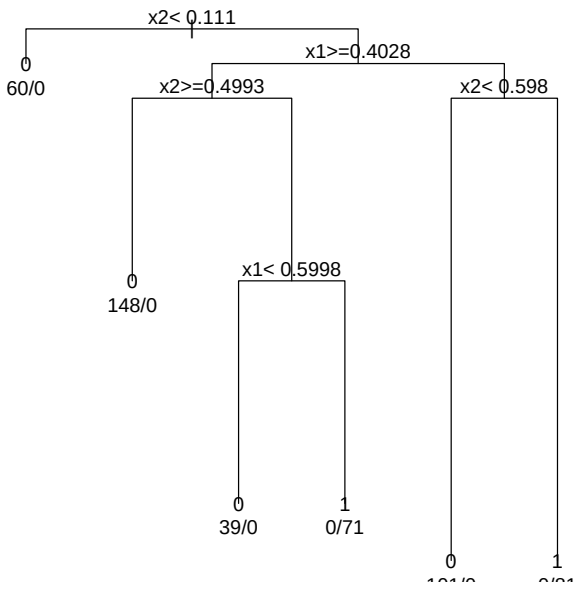


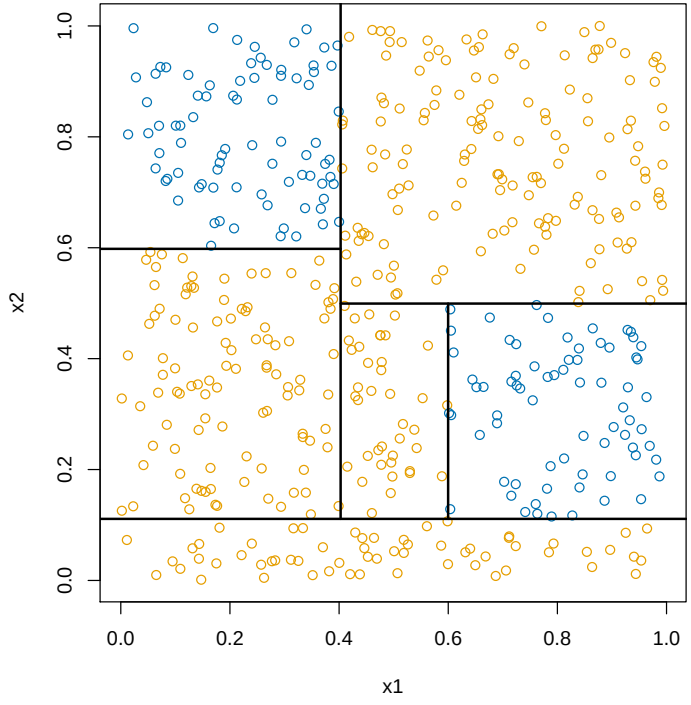
(ISL Figure 8.7)

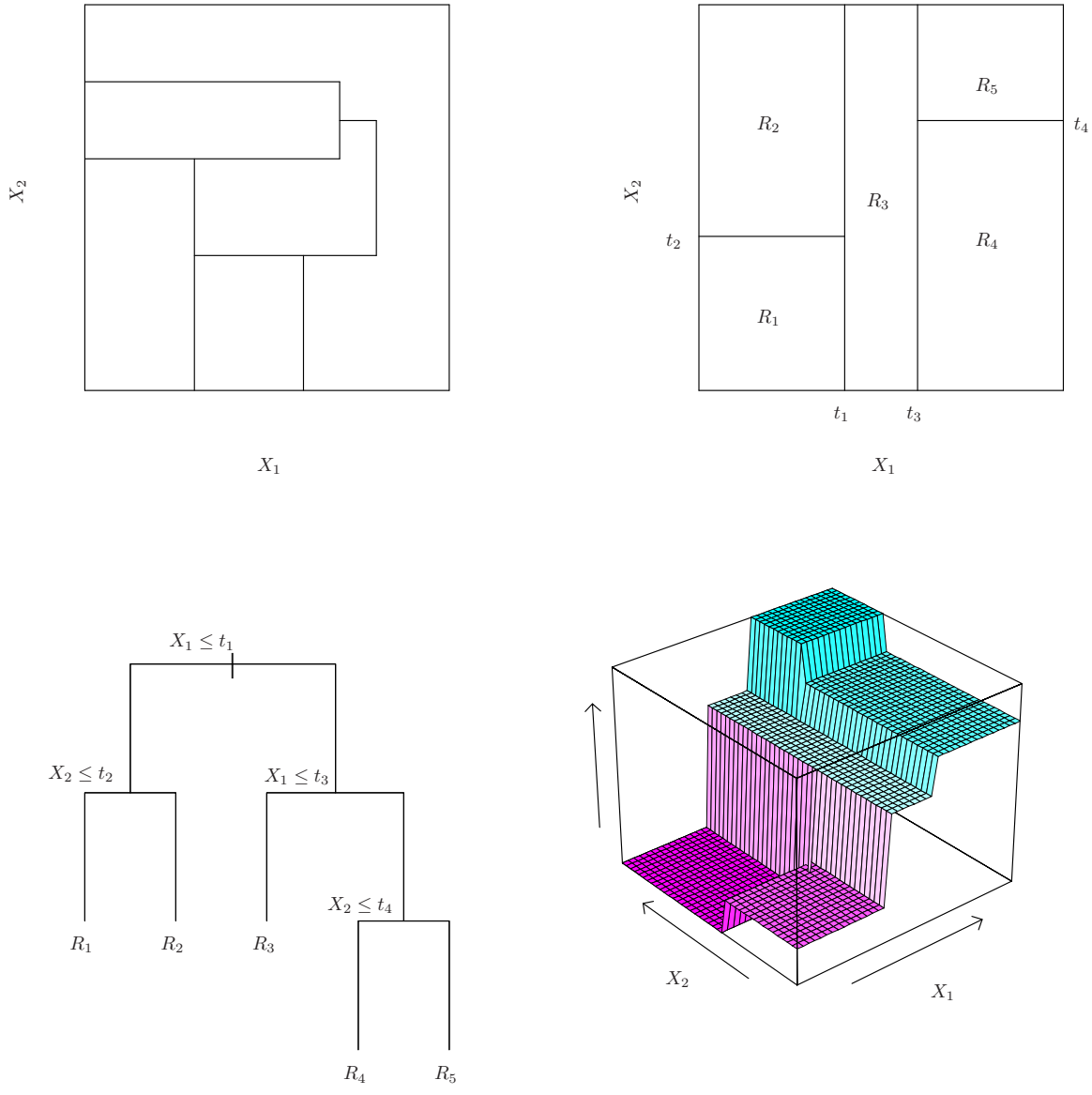
Trees

Because searching over all rectangular partitions is expensive, these methods only consider partitions that can be represented as binary trees

The binary trees are obtained through a simple greedy search







(ISL Figure 8.3)

Trees: Tuning

You learned in ML1 that in terms of bias/variance:

- large/deep trees are:
- small/shallow trees are:

You learned to fit trees by constructing a large tree, and then pruning it back to a reasonable size.

You used cross-validation to determine the appropriate amount of tuning.

Trees

Trees are great because they are simple, interpretable, and do not require strong assumptions.

They can have very low bias, since most functions can be approximated well by a rectangular partition (eventually).

They are even **invariant to monotonic variable transformations**, and they can **capture simple interactions**!

However, they turn out to have poor prediction performance...

How can we fix this?

Bagging

We build low-bias, high-variance trees, and then used averaging of somewhat independent trees to reduce the variance of our method.

To obtain many versions of the tree, we used Boostrapping to generate new version of the data set, and then trained a new tree on each.

This reduced variance while leaving bias essentially unaffected. Deep trees gave low bias and averaging gave low variance.

Bagging Trees

1. For $b = 1, \ldots, B$, we draw n bootstrap samples to form a new data set X^{*b}, y^{*b}.

2. For each bootstrap data set, we train a separate regression/classification tree, $\hat{f}^b$.

3. When we get a new input $x \in \mathbb{R}^p$, we form estimates $\hat{f}^b(x)$ from every tree and combine them to get an answer.

$$\hat{p}_k^{\text{bag}}(x) = \frac{1}{B} \sum_{b=1}^{B} \hat{p}_k^b(x)$$

Then just pick the category with the highest average probability.

Random Forests

Random forests are just bagged trees, plus one more modification.

1. For $b = 1, \ldots, B$, we draw n bootstrap samples to form a new data set X^{*b}, y^{*b}.

2. For each bootstrap data set, we train a separate regression/classification tree, $\hat{f}^b$. When finding each split in the tree, only allow a random subset of variables to be considered!

3. When we get a new input $x \in \mathbb{R}^p$, we form estimates $\hat{f}^b(x)$ from every tree and combine them to get an answer.

This simple twist improves the performance of the resulting ensemble.

Intuition for Random Forests

In general, random forests are approximating $\mathbb{E}(Y|X)$ as an average of many piecewise constant functions over rectangles.

Why subset variables at the splits?

The subsetting also serves a purpose similar to *dropout*.

If several correlated variables serve similar purposes, the subsetting spreads out the predictive power across those variables, making the estimates more robust.

Estimating Generalization Error

Because of the way our bagged trees are constructed, we get a shortcut to estimating the generalization error.

Usually, we would do cross-validation so that some data points have been held out for testing.

Because our trees are all trained on bootstrap samples, some observations are always missing from each tree!

Estimating Generalization Error: Out-of-Bag Error

Instead of cross-validated error, we use *out-of-bag error*:

1. For each data point, x_i, find all the trees where x_i did not show up in the bootstrap sample.

2. Treat that subset of trees as a new mini-forest, and make a prediction for x_i

3. Average the error of these predictions across all observations.

These are really estimating

Digging into Random Forests

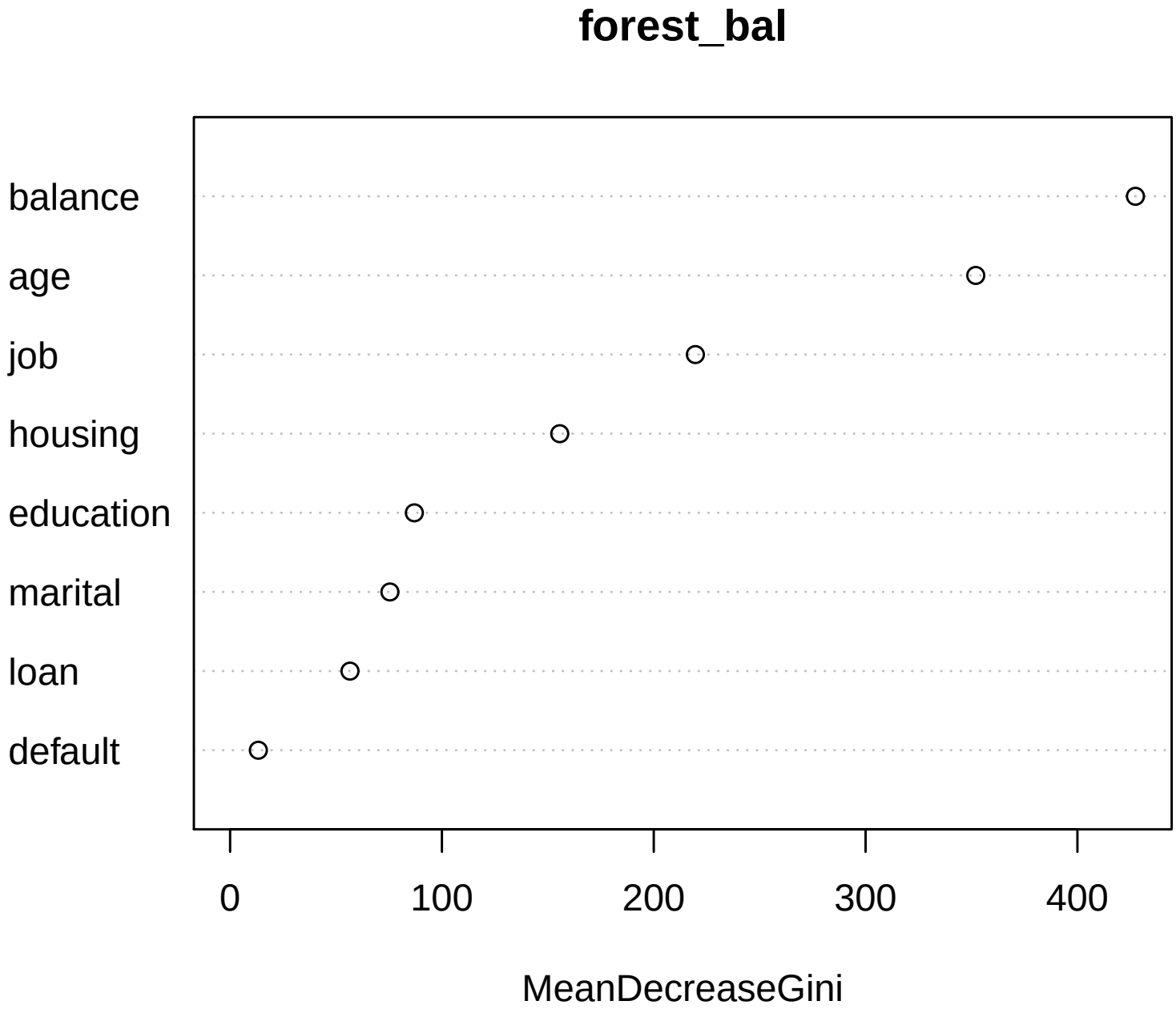
Random forests are one of the first models we've seen where it's incredibly hard to describe intuitively how they are making predictions.

To really see where a guess comes from, you would have to follow a long path down each of the hundreds of trees! How could you even summarize this?

We saw two simple approaches to assess some aspects of random forest function:

- Variable importance
- Partial dependence plots

These same ideas will continue to arise, as explainability continues to be a major problem in ML.


forest_bal
balance
age
job
housing
education
marital
loan
default
0
100
200
300
400
MeanDecreaseGini

Partial dependence plots

Variable importance tells you which variables are useful, but not how they are used.

Partial dependence plots can give some insight into the way that estimates change with each variable.

Suppose we divide our variables into sets $\mathcal{S}$ and $\mathcal{C}$, and we want to assess the effect of the variables in $\mathcal{S}$ on our predictions.

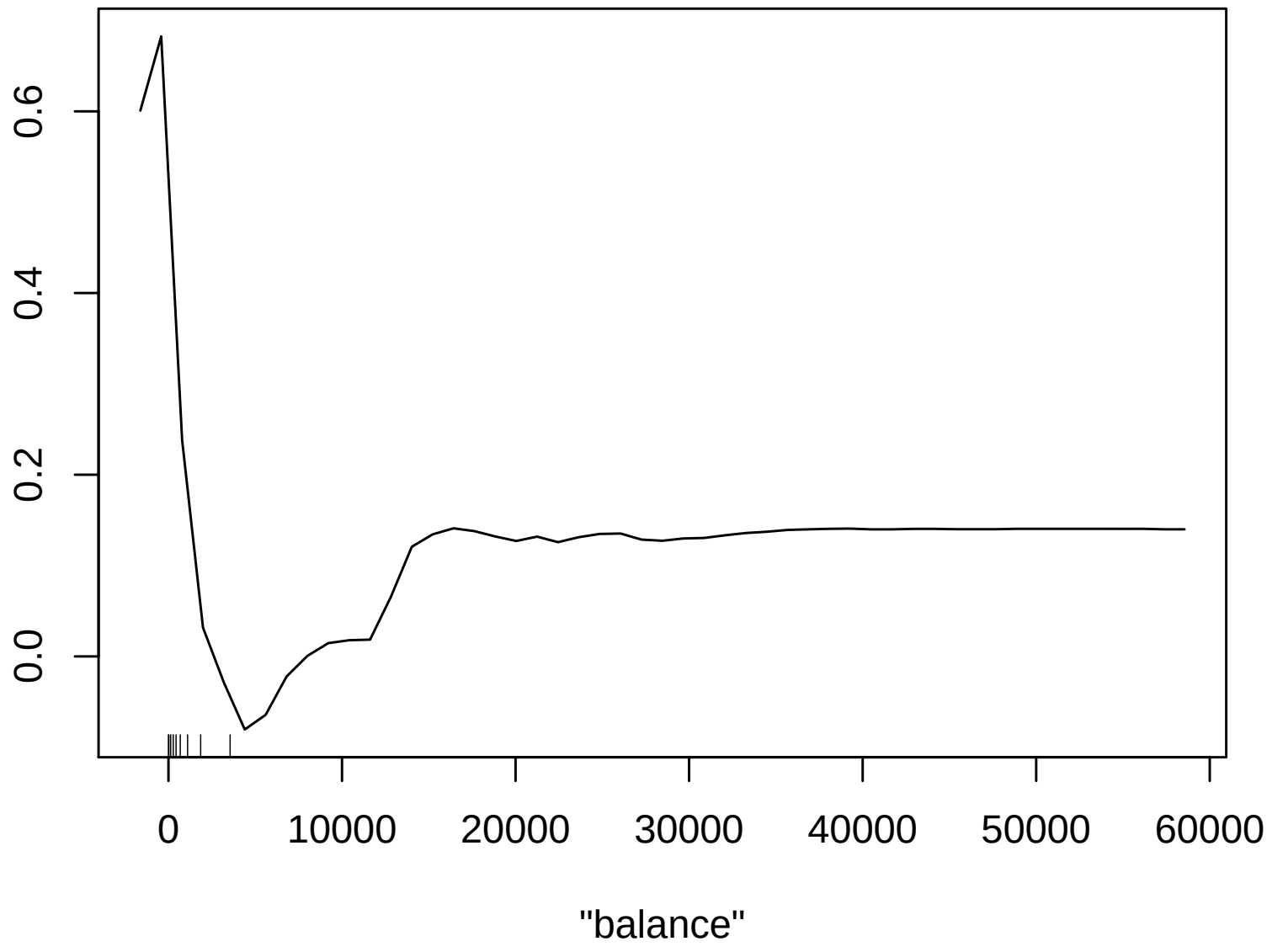
The partial dependence of $f(X)$ on $X_{\mathcal{S}}$ is defined as

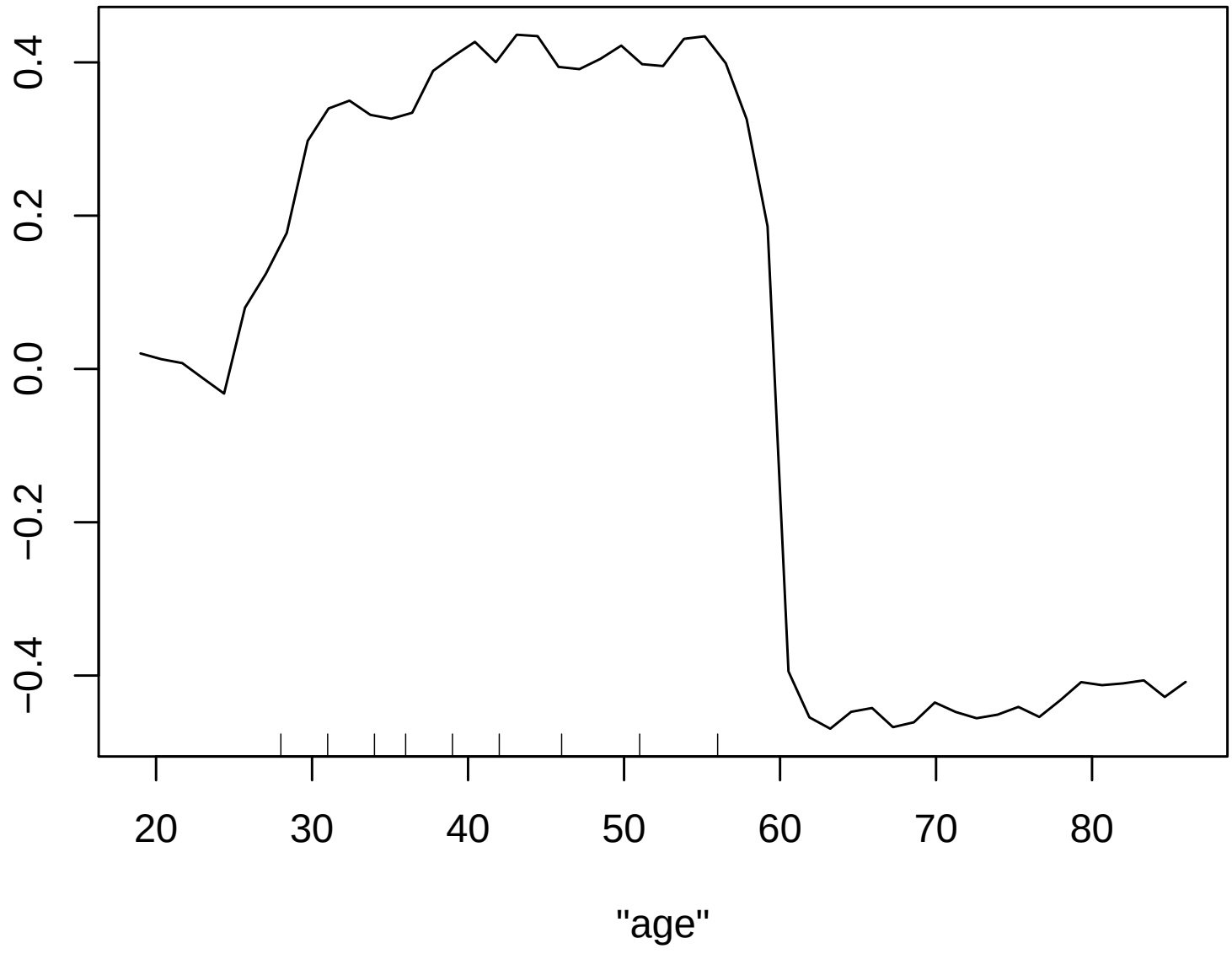
$$f_{\mathcal{S}}(X_{\mathcal{S}}) = \mathbb{E}_{X_{\mathcal{C}}} f(X_{\mathcal{S}}, X_{\mathcal{C}})$$

This can be estimated by

$$\bar{f}(X_{\mathcal{S}}) =$$

where $x_{i\mathcal{C}}$ are just the values that appear in our data.


Partial Dependence on "balance"
0.0
0.2
0.4
0.6
0
10000
20000
30000
40000
50000
60000
"balance"


Partial Dependence on "age"
0.4
0.2
0.0
−0.2
−0.4
20
30
40
50
60
70
80
"age"

From Bagging to Boosting

Random forests give one way to combine a collection of *weak* predictors into a much stronger predictor.

Bagging is not specific to trees. You can use this same resampling approach in many settings.

Boosting is another method to combine many weak predictors into a stronger one.

Boosting

Like Bagging, Boosting is an approach to combining many weak estimators into a powerful estimator.

In *bagging*, we wanted our trees to be as independent as possible, so that the average would have low variance.

In *boosting*, we will build a sequence of simple prediction functions (usually trees). Each function will try to do well on observations that the previous functions did poorly on.

This makes the individual functions very dependent! However, we will see that it can have great performance.

Original Boosting Algorithm (AdaBoost)

Initialize: Weights $w_i = 1/n$

For $b = 1, \ldots, B$**:**

1. Fit classification tree $\hat{f}^b$ to the training data with weights $w_1, \ldots, w_n$.
2. Compute weighted misclassification error:

$$e_b = \frac{\sum_{i=1}^{n} w_i \mathbf{1}\{y_i \neq \hat{f}^b(x_i)\}}{\sum_{i=1}^{n} w_i}$$

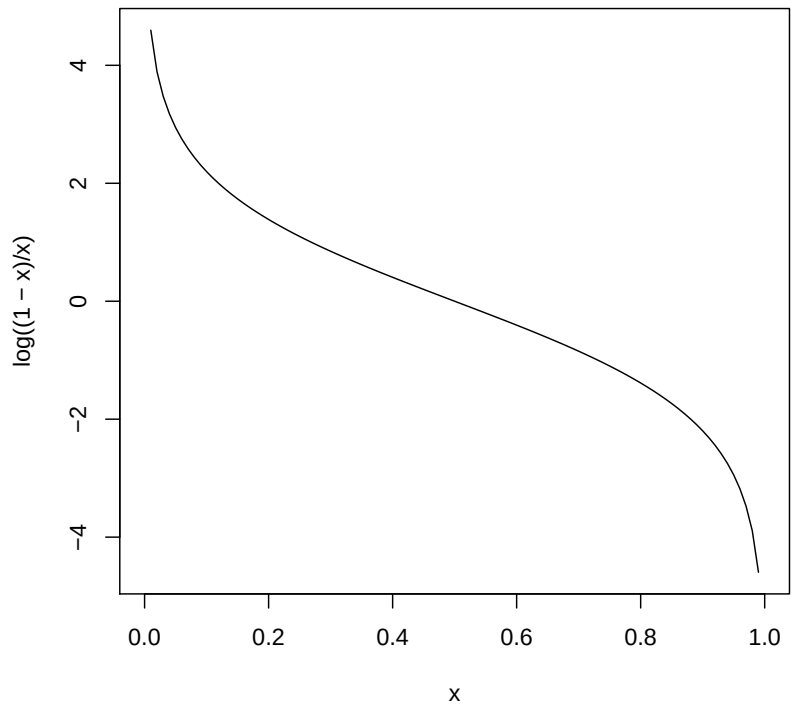
3. Define $\alpha_b = \log \frac{1-e_b}{e_b}$
4. Update the observation weights:

$$w_i \leftarrow w_i \cdot \exp\left(\alpha_b \mathbf{1}\{y_i \neq \hat{f}^b(x_i)\}\right)$$

Result: $\hat{f}(x) = \text{sign}\left(\sum_{b=1}^{B} \alpha_b \hat{f}^b(x)\right)$

AdaBoost: Step 3

$\alpha_b = \log \frac{1-e_b}{e_b}$.



This step stretches out probabilities near 0 and 1. It should remind you of logistic regression.

AdaBoost: Step 4

$$w_i \leftarrow w_i \cdot \exp\left(\alpha_b \mathbf{1}\{y_i \neq \hat{f}^b(x_i)\}\right)$$

If y_i was guessed wrong, multiply weight by

If y_i was guessed right, multiply weight by

If α_b is big, the classifier is did its job

If α_b is big, the weights increase

Adaboost intuition

We have enough for an intuition. Adaboost builds a sequence of trees, each of which tries to classify well on points that were missed by the earlier trees.

The observation weights, w_i, focus later classifiers on difficult points.

The classifier weights α_b capture how well each component classifier does its weighted task, and is used for both

- Adjusting the observation weights
- Weighting the components in the final classifier

But why exactly these weights and functions??

Why this form for AdaBoost?

Suppose that we care about misclassification error, or 0-1 loss. When $Y \in \{-1, 1\}$, we can rewrite this.

$$\hat{f}(x) = \underset{f}{\operatorname{argmin}}\, \mathbb{E}\mathbf{1}\{Y \neq f(X)\} =$$

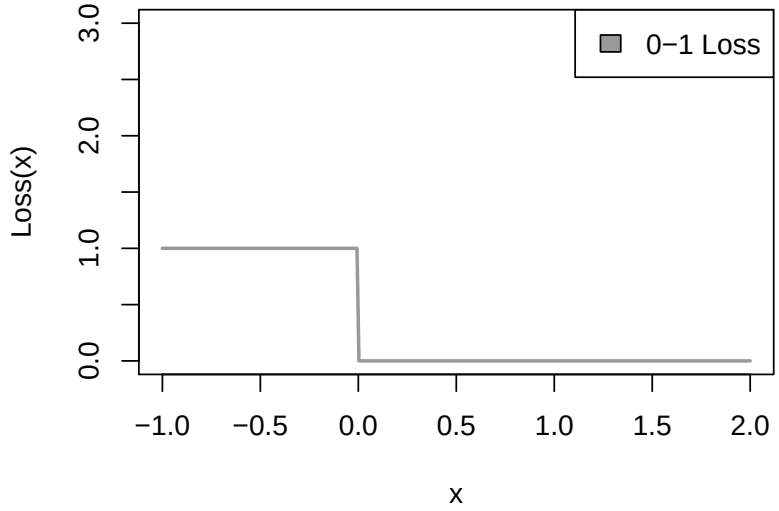
We've constrained ourselves to make $f(X)$ be an additive model built out of simple trees, so the sample version of this problem becomes

$$\hat{f}(x) = \underset{\alpha_b, f^{(b)}}{\operatorname{argmin}} \frac{1}{n} \sum_{i=1}^{n} \mathbf{1}\left\{ y_i \left(\sum_{b=1}^{B} \alpha_b f^{(b)}(x_i) \right) < 0 \right\}$$

Now it just looks like an optimization problem! But how do we optimize it?

Approximating 0-1 loss

It turns out that 0-1 loss is not a very nice function to optimize



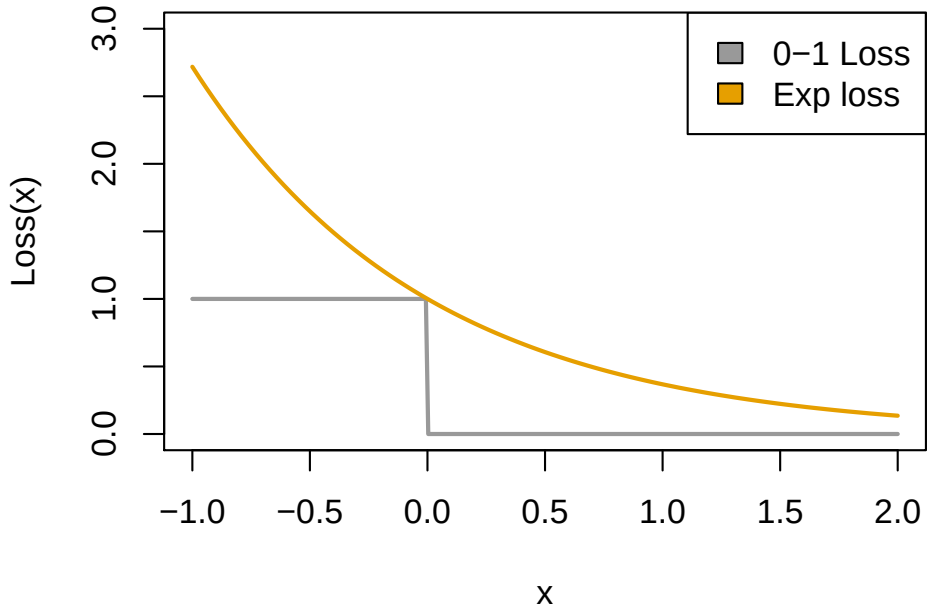
It's not differentiable or continuous. It's also constant except at 0!

Without being able to take a derivative, it's hard to tell what local changes to your function will improve the fit.

With regions of constant loss, there's not a sense of "close to correct," so it's hard to know which points are close to being correctly classified.

Approximating 0-1 loss

We switch 0-1 loss for another function that's easier to optimize and has similar behavior in important ways



This new function is continuous and differentiable and convex, and it is still monotonically decreasing.

If we can make the exponential function small, our 0-1 loss will also be good.

With the new loss

Making that switch, instead of solving

$$\underset{\{\alpha_b, f^{(b)}\}}{\operatorname{argmin}} \sum_{i=1}^{n} \mathbf{1}\left\{ y_i \left(\sum_{b=1}^{B} \alpha_b f^{(b)}(x_i) \right) < 0 \right\}$$

we just have to solve

$$\underset{\{\alpha_b, f^{(b)}\}}{\operatorname{argmin}} \sum_{i=1}^{n} \exp\left(-y_i \left(\sum_{b=1}^{B} \alpha_b f^{(b)}(x_i) \right) \right)$$

With the new loss

We just have to solve

$$\underset{\{\alpha_b, f^{(b)}\}}{\operatorname{argmin}} \sum_{i=1}^{n} \exp\left(-y_i \left(\sum_{b=1}^{B} \alpha_b f^{(b)}(x_i)\right)\right)$$

This is still tricky, so we cheat just a little more. We optimize one $\hat{f}^{(b)}$ at a time while holding the previous functions fixed, and never come back.

$$\underset{\alpha_b, \hat{f}^b}{\operatorname{argmin}} \sum_{i=1}^{n} \operatorname{Exp}\left(-y_i \cdot \left(\sum_{k=1}^{b-1} \alpha_k \hat{f}^k(x) + \alpha_b \hat{f}^b(x)\right)\right)$$

This is a *greedy* algorithm. It seeks immediate rewards, rather than optimizing the overall objective.

Oddly enough, optimizing this objective leads to the entire AdaBoost algorithm with all the weights and transformations!

[If we have time, show this by hand]

Empirical risk minimization

This general pattern:

1. We want to minimize:

$$\mathbb{E}\mathbf{1}\{Y \neq f(X)\}$$

2. And so we actually try to minimize its empirical version:

$$\frac{1}{n}\sum_{i=1}^{n}\mathbf{1}\{y_i \neq f(x_i)\}$$

3. But we can't even do that for classification! So we introduce a nicer loss L and minimize

$$\frac{1}{n}\sum_{i=1}^{n}L(y_i, f(x_i))$$

The first two steps are known as *empirical risk minimization*. The last step almost always follows for classification.

Adaboost final summary

Adaboost is just

1. Empirical risk minimization with an exponential loss

$$\frac{1}{n}\sum_{i=1}^{n} e^{-y_i f(x_i)}$$

2. Assuming an additive model of trees:

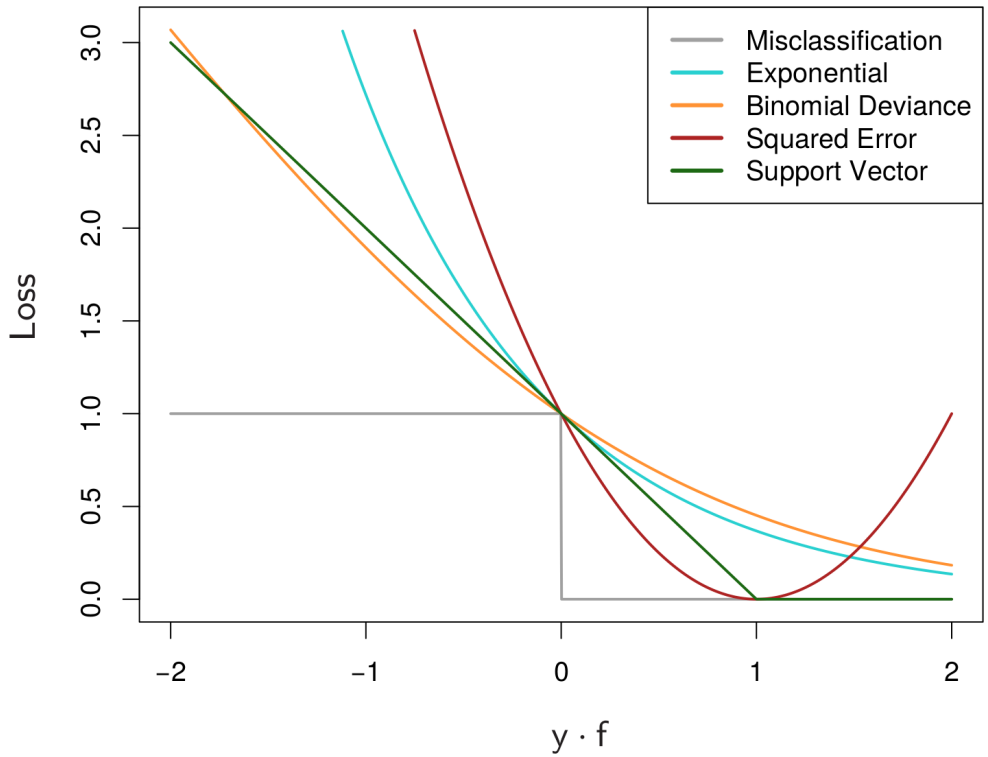
$$\hat{f}(x) = \sum_{b=1}^{B} \alpha_b \hat{f}^{(b)}(x)$$

   with $\hat{f}^{(b)}(x)$ constrained to be a tree.

3. And greedy, stepwise optimization

$$\underset{\alpha_b, \hat{f}^b}{\operatorname{argmin}} \sum_{i=1}^{n} \operatorname{Exp}\left(-y_i \cdot \left(\sum_{k=1}^{b-1} \alpha_k \hat{f}^k(x) + \alpha_b \hat{f}^b(x)\right)\right)$$

Improvements: Why exponential loss?



(ESL Figure 10.4)

There are many losses that approximate 0-1 loss. Some can give dramatically better performance in certain settings.

How do we minimize for other losses?

It turns out that the exponential loss leads to a particularly simple algorithm, while the others do not. The optimization becomes harder, and the updates lose their beautiful form.

At step b, we want (for a tree $\hat{f}^b(x)$)

$$\underset{\alpha_b, \hat{f}^b}{\operatorname{argmin}} \sum_{i=1}^{n} L\left(y_i, \left(\sum_{k=1}^{b-1} \alpha_k \hat{f}^k(x) + \alpha_b \hat{f}^b(x)\right)\right)$$

What if we think instead about

$$\underset{\delta \in \mathbb{R}^n}{\operatorname{argmin}} \sum_{i=1}^{n} L\left(y_i, \sum_{k=1}^{b-1} \alpha_k \hat{f}^k(x) + \delta\right)$$

Gradient Boosting

$$\underset{\delta \in \mathbb{R}^n}{\operatorname{argmin}} \sum_{i=1}^{n} L\left(y_i, \sum_{k=1}^{b-1} \alpha_k \hat{f}^k(x) + \delta \right)$$

Gradient Boosting

At iteration b, the gradient $g_b \in \mathbb{R}^n$ of the loss function is found. The new classifier $\hat{f}^{(b)}$ is then fit to approximate this gradient vector well. Hence the name: *Gradient Boosting*.

Conveniently, there have been several great implementations of gradient boosting developed recently. The most popular is `xgboost`, which luckly exists in both R and python. Two other popular ones are `catboost` and `lightgbm`.

Other Improvements

Thinking of boosting as optimization automatically leads to other improvements

- *Shrinkage:* Only add a small amount of each tree at a time. Like taking smaller steps. Provides regularization. Use $\nu\alpha_b\hat{f}^b$ with $\nu \in [0, 1]$.
- *Subsampling:* Each step only sees a fraction, η of the data. Gives improvements to variance (and thus performance), as well as speed!

Boosting discussion

In bagging/random forests, we use deep trees to stay unbiased, and then rely on averaging to reduce variance.

In boosting, we tend to use very simple estimators, like very shallow trees. These have low variance, but are high bias. Because the sequence of trees can adjust for previous errors, the bias can be corrected!

The shallower trees can also lead to computational speedups in evaluation, since you don't have to evaluate 500 deep trees.

Think of the depth of your trees as allowing interactions. Two splits let you interact two variables. You'll often find that you want somewhere between 2 and 10 splits in your trees.

Boosting discussion

Boosting is one of the most powerful classical methods. When well-tuned, it can beat random forests and most other classifiers.

However, it requires more tuning than random forests.

Tuning parameters:

- Number of trees, B
- Amount of subsampling, η
- Amount of shrinkage, ν
- Number of splits in each tree (or proxies for this)
- (Actually many, many more)

You will find that random forest are difficult to badly overfit. In boosting, choosing B too large can overfit. Choosing it too small can give a bad classifier.

B is often the main tuning parameter. The others can often be tuned more roughly, since it doesn't matter quite as much.

Ensemble Learning

One last comment on combining classifiers.

In boosting and bagging, we saw two approaches to combining many weaker predictors into a much stronger predictor

This combination is called an *ensemble*. There are many general approaches to building ensembles.

Ensembles can combine methods of different types with different strengths, so that each can perform well in cases where it is strong.

Many contests are won in this way. In practice, there are sometimes practical concerns about complexity and speed.

How can we combine classifiers?

Suppose that I build M different classifiers on my data, $\hat{f}_1, \ldots, \hat{f}_M$. How can I combine their guesses?

- Voting, weighted voting
- Averaging
- Stacking:

Stacking